Directing a multi-machine computation with Claude Code: the method behind a computation of Choptuik's echo period

Nathan Moinvaziri[1]

[1] *Independent researcher*

(Dated: November 2026; unreviewed draft)

I describe the method by which I carried out a numerical research computation with Claude, Anthropic's model, running in Claude Code on several machines. The computation is the echo period $\Delta(D)$ of Choptuik's critical solution in D spacetime dimensions, which over 19 days reached down to $D = 3.04375$ and up past $D = 200$. It began in a two-day chat, which went from a general question to a reproduction of Choptuik's result and then to a search for a formula, before it moved into a repository and Claude Code. I set the goals, the standards of evidence and the decisions. One coordinating session wrote all the code and queued numbered tasks. Runner sessions on the other machines ran the stated commands and reported through an append-only file in the repository, and the coordinator gave reviews, searches and builds to subagents. I describe the division of roles, the coordination protocol, how tasks are specified, the rules of evidence and their controls, how runners and subagents were used, and the patterns that unblocked stuck work, each with a short illustration from the records, and I state the limits of the approach.

I. INTRODUCTION

When the collapse of a massless scalar field is tuned to the threshold of black-hole formation, the evolution approaches a universal solution that repeats itself on ever smaller scales, each echo a copy of the last shrunk by e^{Δ} [1]. The repository `choptuik-large-d` [2] computes Δ as a function of the number D of spacetime dimensions, and its physics is in two companion drafts [3, 4]. Each point is a periodic boundary-value problem solved by Newton's method from a prediction made at the last point. At large D the mass mode amplifies errors at the horizon by a gain $G \approx 0.7^{-(D-3)}$, so the arithmetic must carry more digits as D grows, from double-double [5] (about 32 digits) through quad-double [6] (about 64) to octo-double (about 126). At low D one point costs 4 to 10 hours.

This paper is about the method, not the physics. The project began in a chat on claude.ai on 17 and 18 October 2026 (Sec. II). From 19 October to 6 November I directed Claude sessions running in Claude Code [7] on five machines. Claude produced the code under my direction. Downward, the lowest converged point is $D = 3.04375$. Upward, a chain of continuation steps passed $D = 200$ at 16:04 UTC on 6 November and stood at $D = 206.77$ at 18:47 UTC, still climbing.

The sources are the chat where the project began [9], the coordination log `comms.txt`, the git history, the notes in `notes/`, the raw-data repository [8] and the transcripts of the Claude Code sessions on the coordinator machine. The other machines' side is known from their entries in the log. Times are UTC, and every number and quotation is listed with its source in `paper/ai/SOURCES.md`.

II. HOW IT BEGAN

The chat began on 17 October at 23:24 with general questions, such as "*Are you able to make new discoveries about physics?*" I asked for "*a toy numerical reality*" (23:42), and Claude chose a collapse simulation in general relativity. From 177 runs it reproduced Choptuik's mass scaling, with an exponent of 0.377 and 0.378 for two pulse shapes against the accepted 0.374, and an echo period $\Delta \approx 3.30$ against 3.445. It named its own limit, a fixed grid that resolved only about one echo.

The turn came with "*Explain the collapse numbers*" on 18 October (16:43). The answer said that both numbers are eigenvalues of exact problems with no known formula, and it proposed a plan with a literature check first. The check changed the target, since a 2026 paper had already computed Δ for continuous D between 3.05 and 5.5 (see [3]). Claude then computed the critical solution directly, as a periodic problem rather than a simulation. At 63 Fourier modes it gave $\Delta = 3.4454523972$ in four dimensions, which agrees with the best published value, 3.445452402, to nine digits, and it went on to scan D.

Four prompts that evening pushed toward a formula: "*Yes continue to find the formula you are doing great don't give up. It exists*" (20:24), "*Could it be a sum?*" (20:41), "*What if is a combination of a small sum and large sum?*" (21:41) and "*Where is the formula?*" (21:45). They shaped the method in two ways. First, each suggested form was turned into a fit and tested on dimensions it had not seen, as when a fit to $D \le 8$ predicted the computed values out to $D = 10.1$ to within 2×10^{-5}. That habit later became the held-out and planted-truth tests of Sec. VI. Second, the

search produced a structural finding. The solutions that compete with the smooth one behave like $(1-x)^s$ near the horizon, with s measured to grow like $0.6\,(D-2)$, so the condition that fixes Δ weakens rapidly with D and no plain series in $1/D$ can fix it. Claude called the expected form a transseries, and the idea became Proposition 1 of the later work, "the $1/n$ expansion never fixes the period". The fitted formulas themselves did not last. The first one for Δ, fitted on data below $D = 11$, fell like $(D-2)^{-1/4}$, while the later data give a finite limit, $\Delta_\infty = 1.643893746$ [3].

A formula needed many precise points, so speed became the problem. On 19 October I asked "*can we speed up the runtime by using SIMD?*" and "*Does it make sense to rewrite in Rust?*" (02:02), and then "*With a subagent make a purpose build Rust build and measure speed.*" (02:24), the first subagent of the project. The chat ran in a cloud container with two virtual cores. I put what the chat had produced into a repository, whose first commit, "Import the DSS residual kernel", is dated 19 October 03:53, and told Claude so at 04:27. At 04:35 I linked the chat to my laptop, "*You are now local on my computer. It has AVX512 too which is nice. Continue our work.*" By 08:00 the first entries of `comms.txt` passed between a cloud session and the laptop, and the method of the following sections grew from that pair.

III. ROLES

I set the goals, from "*continue to D = 200*" on 27 October to "*we can push past D=200 and go as large as we can*" on 6 November. I set the standards of evidence and chose between options, by short prompts and by answering the coordinator's multiple-choice questions. I also acted on the machines myself. I merged and released the arithmetic library `multidouble` [10], and I woke, plugged in and set up machines.

The coordinator was one Claude Code session at a time, on a cloud machine until 26 October and on the Mac after it. It wrote all the code in its own worktrees, merged results into main, wrote the notes and queued numbered tasks, T1 to T135.

The runners were one Claude Code session per other machine. They ran the commands of their tasks, posted results and pushed raw output, and they did not edit code. That rule came from my prompt of 26 October, "*make a note that the other machines are running a lower model than you are, so it is best for you to write any code and not them.*" The Mac runner's transcripts record Claude Sonnet 5.5 from that day.

Subagents were sessions the coordinator started for one bounded task, each returning a single report.

TABLE I. The machines, by their names in `comms.txt`, with the entries each posted (headers counted at commit 07eff89). The laptop posted as `local` until 24 October.

name	machine	role	entries
`cloud`	cloud container, 2 cores, 6.27 GB	coordinator to 26 October; then the coordinator's name on the Mac	293
`win`	Windows laptop, 4 cores and 8 threads, 31.8 GB	runner from 19 October: low D, builds, second paths	318
`mac`	Mac, Apple M5, 10 cores	runner from 24 October to 5 November; coordinator from 26 October	221
`win2`	Windows desktop, 20 cores, 95.6 GB	runner from 1 November: the large-D chain, low-D legs	134
`macm3`	Mac laptop, Apple M3, 8 cores, 24 GB	runner from 6 November: a second path past $D = 193.77$	8

IV. THE COORDINATION PROTOCOL

One log. All sessions talked through `comms.txt` in the code repository (Fig. 1). It is append-only. Each entry has a header such as `== 2026-11-06 13:34 UTC cloud to win2`, with the time read from a clock, and a writer pulls, appends, commits only that file and pushes. It held 974 entries by 18:26 on 6 November. When the coordinator moved to the Mac, I chose to keep `cloud` as its name so the runners' watchers needed no change.

Watchers. A runner waits with `comms_watch.py`, which fetches the repository every 60 s and exits when an entry addressed to its name arrives, waking the session. After 110 minutes it exits anyway, because Claude Code stops a background command after two hours, and the session restarts it. It costs nothing while it waits, and it replaced polling loops.

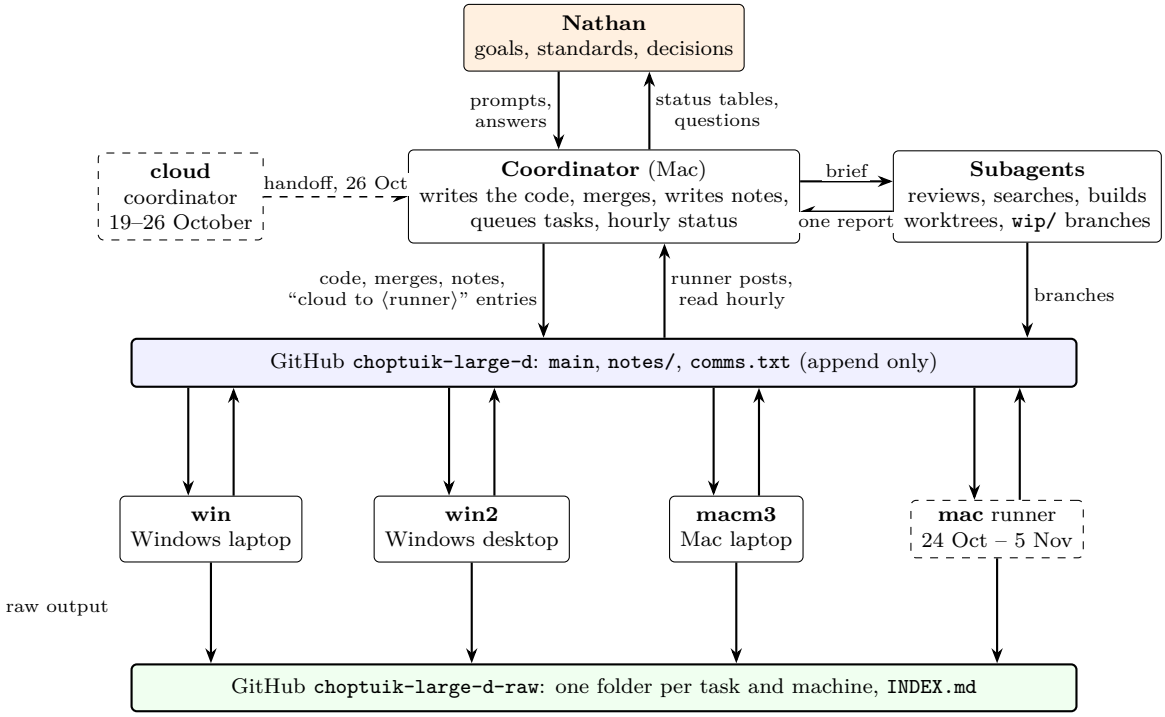


FIG. 1. The participants and how they were connected. I spoke only to the coordinator, and the coordinator spoke to the runners only through `comms.txt`. Each runner's watcher wakes it when an entry addressed to it arrives (down arrows), its posts carry its processor and memory use (up arrows), and its raw output goes to the raw-data repository. Dashed boxes are sessions that ended.

Raw output goes to a separate repository, one folder per task and machine, each with a README whose first line says what it holds, indexed by `INDEX.md`. The long-running scripts refuse any other output path, and `raw_check.py` fails when a folder a post names is not on GitHub. The rule followed a loss. On 26 October the coordinator found that the per-point records of 757 early points had stayed on the cloud machine's disk and were gone, though their values survived. I then asked "*from now on all agents should store the raw output in git somewhere different.*"

Status. A recurring prompt asked the coordinator every hour to fetch, read the new entries, check the branches and report. It fired 308 times. I shaped the answer into one table row per runner and task, naming the problem each task serves and an ETA in hours, and I asked the runners to report processor and memory use so idle capacity would show.

V. HOW A TASK IS SPECIFIED

A task entry states its commands, its thread count, its stop conditions and what to push (AGENTS.md). A shortened example, the entry that sent the octo-double chain to win2 on 6 November, shows the form.

```
1. win2, T129 (the chain to 200 in octo-double): ... below normal, 18 threads:
     anm_run.py ..\..\choptuik-large-d-raw ..\..\choptuik-large-d-raw\T129-win2\chain
       --state T127-win2/chain2/anm_pin3_D175.5246.npz --kfreeze 3 --N 24 ... --jet_o 1
   ... Stop at D_max, on a degenerate series, when two steps in a row need a of 0.125
   or less, or after 10 h. Push T129-win2/ at each post.
```

Four further habits made tasks robust.

- *Pilots first.* A long run starts with a short run of its hardest case and a written stop condition. The switch of the chain to octo-double followed a two-step pilot run beside the old chain.
- *Rules the runner applies without waiting.* An entry can arm a rule, such as "if the log gains no line for 20 minutes while one core is busy, take one py-spy dump, kill the tree, restart from the last state with h_max halved (0.25, then 0.125), and post. Do not wait for me." Another told win2 to "Switch it yourself, without waiting for me, when an accepted stall passes 1e-26, or when chain4 stops on three halvings".

- *Second paths.* A new value counts once another run, usually on another machine, reaches the same D. On the chain, 18 such runs give the error column of 66 rows, with differences from 1.9×10^{-32} to 2.9×10^{-19}.
- *Reporting, not fixing.* A runner that sees a mismatch posts it and continues or waits as the task says. "Do not edit code; on an error, stop and post it" closes eleven entries.

VI. RULES OF EVIDENCE

AGENTS.md states the rules, written on 21 October and partly taken from the verification protocols of BootLoops [11]. Every check must be able to fail, so each check is broken once on purpose and the broken version stays runnable. Planted truth comes before real data. Every error bar says what it is. Independent checks share none of the machinery under test. Discrepancies are reported, not quietly fixed, and thresholds are not loosened to pass. Every quoted number is tied to a file by `check_numbers.py`, which also checks that each test rejects a value shifted by one unit. Table II shows the rules in the late large-D work.

TABLE II. Checks and their controls in the late large-D and low-D work. Each control was run and failed as it must.

check	pass	control, which must fail	source
re-seed of a stored prediction (`checkq`)	converges, 2.25×10^{-37} at 153.43	the previous rule stops at 3.60×10^{-23}, exit 1	QUAD_DOUBLE 13.26–27
bit identity of a new build	28 of 28 jet outputs equal	a mutated constant, 28 of 28 differ	OCTO_DOUBLE 3
octo-double series against two quad-double forms	within 30 times their spread at every order	the double-double series, off by up to 10^{25}	OCTO_DOUBLE 3
octo-double corrector from a planted start	6.86×10^{-42}, Δ to 1.18×10^{-34}	one term through the quad-double LU, stops at 3.39×10^{-12}	OCTO_DOUBLE 5.4
every value of Δ against its source	2445 rows equal	one row moved by one ulp, exit 1	QUAD_DOUBLE 13.25
low-D planner probe, smoke test	every case plans, exit 0	a cap of 0.001 s caps all, exit 1	LOW_D 16

A control that does not fire leaves its check untested. That happened once, in the octo-double kernel's tests, where a planted change landed in a part that quad-double still holds. The planted root was moved and the criteria were kept.

VII. RUNNERS AND SUBAGENTS

Runners built the solver and checked each build bit for bit against stored references, ran the chains and the low-D legs, and posted per-step numbers. Their value lay as much in what they flagged. Table III lists the failure modes the records show, with an example and the remedy that followed. The coordinator's own errors appear there too. Of the fifteen coordinator errors the records show, eight were caught by another Claude session, five by the coordinator's own checks and two by me.

One lesson about authority came from the watcher. When I asked the coordinator to tell the laptop to restart its own watcher after a build, the instruction relayed through the log did not count as mine, and the laptop's session restarted it only when I asked in that session.

Subagents. The coordinator's transcript records 29 subagent launches for this project. Six were reviews and audits, among them the readability of the papers, the README and the quoted numbers (17 findings, 13 fixed). Six were searches, such as methods for continuing to large D. Six were analyses, such as the low-D code and a formula for $\Delta(D)$. Ten were builds, among them pull requests to `multidouble`, re-seeding of the horizon data and the octo-double corrector, and one wrote this paper. The coordinator used them where independence mattered (a reviewer that did not write the work), where a wide search would fill its own context, and where a build of hours could run beside its work. My standing instruction is not to use subagents without asking and naming their number, and 26 of the 29 followed a prompt of mine asking for one, such as "*run two subagents if u need to*". From 4 November, 22:45, every brief named a git worktree and many a backup branch `wip/`⟨`topic`⟩. The coordinator checked each report before relaying it. When it merged the octo-double series, its smoke test showed that the merged driver imported the new helpers at module level, which would have broken every other path on the runners' older packages, and it moved the

TABLE III. Failure modes of the multi-session setup, with examples from `comms.txt` and the remedy adopted.

failure mode	example	remedy
a command that does not run as written	a relaunch of a stopped run exited at once (T82); a check's arguments were wrong (T126, T133). The runner found the cause in the code and said what it did instead.	a correction entry; runners describe any deviation
a wrong expectation in the task	a control expected to fail one check failed three (T104b); a check compared a computed plan across operating systems (T112). The runner reported and did not fix.	the coordinator corrects its text or the check
a cause misread	win2 read re-seed failures as a platform difference (T118, T127); the coordinator replayed the stored inputs and found the code	store every input a failure depends on
a timing rule that misfires	a 30-minute hang rule shorter than one 31-minute series (macm3); a slow LU read as stuck after 3.7 h (T132); a sleeping Mac read as oversubscribed	rules tied to processor use; phase timers; a stack dump before a kill
a watcher that stops	Claude Code stopped the laptop's watcher under memory pressure, once for 56 hours that the status missed, until I asked "*the win machine is not doing anything?*"	silence shown in the status; restart on the author's word in that session
a machine that sleeps	the Mac hibernated for 7 hours, and once ran out of battery; the laptop slept on battery	`caffeinate` tied to runs; awake-time accounting; an always-on desktop (win2)

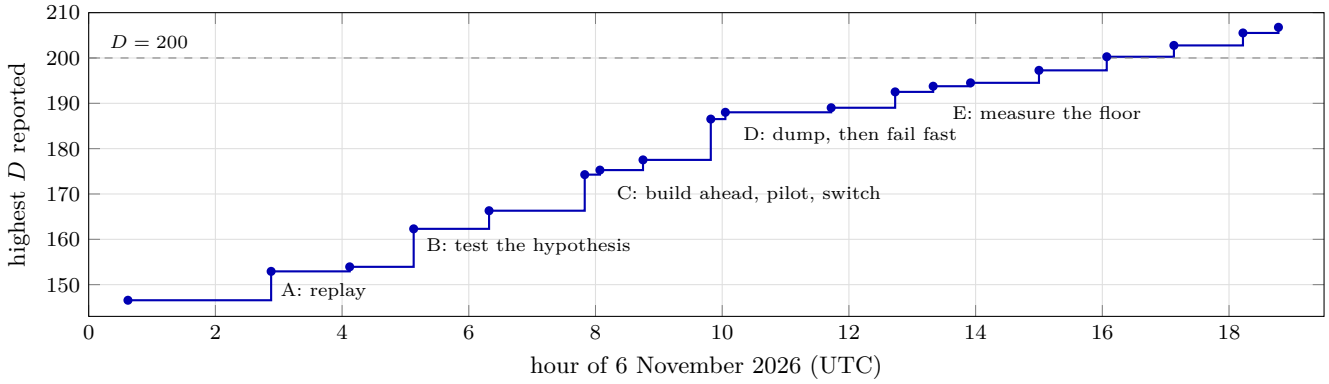


FIG. 2. The highest D in the runners' posts on 6 November, with the walls at 152.93, 162.31, about 171, 188.02 and 193.77, labelled by the pattern of Sec. VIII that passed each. The chain ran on win2, and the fixes were made on the Mac.

import. A subagent's code analysis also corrected one of the coordinator's diagnoses, a stall at low D that was in fact a stale checkpoint.

VIII. UNBLOCKING STUCK WORK

The large-D chain met five walls on 6 November (Fig. 2). Each was passed by one of a few patterns.

A. Replay the failure on its stored input. At 152.93 every re-seed of the horizon data failed. Twenty-two minutes after the stop I wrote "*we need to keep going above 150*", and the coordinator replayed the failing step on the Mac instead of waiting for a queued replay. Within 34 minutes it had found two defects, a polish that gave up after six steps and a corrector that stored an unprojected state, and fixed both with a control that fails on the old rule. From then on the runs saved their predictions, so later failures could be replayed offline on the real case.

B. Test the hypothesis before acting on it. At 162.31 the coordinator's first reading was an imprecise Jacobian, which would have made the coming octo-double series useless there. A cheap test showed the quad-double and octo-double Jacobians equal to 1.2×10^{-36}, and the stored predictions then let candidate rules be compared on the failing input until one converged (`notes/QUAD_DOUBLE.md` 13.27).

C. Build the next precision ahead of need, pilot it, then switch. On 5 November I wrote "*yes add octo-double jets*

TABLE IV. Prompts that moved stalled work, verbatim, and what followed.

when	prompt	what followed
18 Oct 16:43	"*Explain the collapse numbers*"	A reproduction became a direct computation, checked against the literature, then a scan in D.
18 Oct 20:41	"*Could it be a sum?*"	Suggested forms were fitted and tested on unseen D, and the search found why no plain $1/D$ series fixes Δ.
27 Oct 06:56	"*just make the quad-double solver, stop delaying.*"	The quad-double solver was built that morning, with precision tests.
2 Nov 22:40	"*just jump to 100*"	Jumps of every size failed past 79.3, which located the wall in Newton's basin rather than the step rule.
3 Nov 19:21	"*have a subagent scourer the internet and scientific papers for ways continue large D*"	A literature subagent found high-order Taylor continuation in D [13], which became the chain.
5 Nov 02:34	"*have a subagent build the real thing and not a toy.*"	Re-seeding of the horizon data moved from a model into the real solver.
5 Nov 22:25	"*yes add octo-double jets in hp engine, we don't need to run it until it is needed.*"	The octo-double series was ready before the quad-double one failed (pattern C).
6 Nov 03:15	"*we need to keep going above 150*"	Immediate replay and fix of the stall at 152.93 (pattern A).
6 Nov 13:28	"*we can push past D=200 and go as large as we can*"	The chain's limit went to 1000 with an explicit stall rule (pattern E).
6 Nov 13:54	"*can we queue it on a machine that is not busy?*"	The low-D probe moved to the laptop, displacing a lower-value task.
6 Nov 14:25	"*i can spool up another mac M3 laptop if you think it will help make the total work faster.*"	macm3 was set up and bit for bit with the Mac's build in 75 minutes, then ran a second path.

in hp engine, we don't need to run it until it is needed." When a runner flagged the quad-double series losing digits near 171, a two-step pilot showed the octo-double series healthy and Δ unchanged in 21 printed digits, and the switch took one entry. The octo-double corrector followed the same path. It was built by a subagent with a planted check and a failing control, merged at c496ac8 on 6 November (`notes/OCTO_DOUBLE.md` 5–6), and handed to the runners as a rule to switch on their own.

D. Dump the stack before killing, then fail fast. Above 188.02 a step ran on one core for more than an hour with no output. The runner did not kill it but asked, the coordinator asked for a py-spy [12] dump first, and the dump put the hang somewhere other than where the coordinator had guessed. A fail-fast on that path and an armed hang rule followed.

E. Measure the floor and accept it explicitly. At 193.77 the quad-double corrector stalled at 1.0 to 1.1×10^{-28} against its tolerance of 10^{-28}, and halving could not help. After my prompt "*we can push past D=200 and go as large as we can*", the coordinator measured the floor on stored states and added a rule that accepts a stall below 10^{-24}, with a flag at 10^{-26} and a second path on macm3. This rule weakens the convergence test, and I record it as such. Its measured effect on Δ is 2×10^{-18} to 1.1×10^{-17} so far, and the runners switch to the octo-double corrector when the flag fires.

F. Planted truth for a diagnostic. At low D the planner froze at $D = 3.04375$. Before reading the probe that separates a bad prediction from a bad planner, its procedure fixed a planted case that must reproduce the known freeze and a control that must fail (`notes/LOW_D.md` 16). The same episode shows the cost of a wrong rule. The coordinator told the laptop that no output after 3.7 hours meant stuck, the laptop's dump showed a slow LU instead, and the probe now times each phase. That question was still open at the time of writing.

Table IV lists the prompts of mine that moved stalled work, from the chat of Sec. II on. Most were short. Their effect was to replace deliberation by an action, to remove a limit that the work had taken for granted, or, in the chat, to keep a search going and suggest a form to test.

IX. THE RECORDS

Table V counts the records the method left. The log, the commits and the raw-data folders together let a result be traced from its entry to its run and its output.

TABLE V. The records, at commit 07eff89 of the code repository and d0f3311 of the raw-data repository.

quantity	value
chat on claude.ai	17 October, 23:24, to 19 October, 06:18; 62 prompts, a few on side projects
period of the repository	19 October 2026, 03:53, to 6 November, 18:26; 19 days
commits on main	2227, of which 954 are entries of `comms.txt`
entries in `comms.txt`	974: coordinator 293, Windows laptop 318, Mac 221, Windows desktop 134, macm3 8
tasks	T1 to T135
raw-data repository	124 folders (laptop 61, Mac 44, desktop 17, macm3 2), 548 commits
values of Δ in `hp/data/delta_all.csv`	2445 rows, from $D = 3.04375$ to 203.27
coordinator session on the Mac	334 prompts, 308 hourly status prompts, 29 context summaries, 30 subagent launches

X. LIMITATIONS AND LESSONS

No expert has reviewed the work. I have not checked the mathematics myself. The checks of Sec. VI make the work testable, and they do not replace a reviewer.

The records are incomplete. The cloud sessions of the first week left no transcript on the coordinator machine, and the runners' transcripts stayed on their machines. The per-point records of 757 early points are lost.

The error bars are of several kinds. Differences between runs measure the path dependence of the held harmonics, about 10^{-20}, and the stall rule, 10^{-18} to 10^{-17}. Neither includes the bias of the held harmonics, 10^{-8} to 10^{-7} at $D = 79$ to 86, which the README states separately.

Wrong instructions cost the most. The runners carried out tasks as written, so a wrong rule cost what it said. What corrected them each time was a measurement, such as a stack dump, a replayed input or a phase timer.

Machines need a person. Sleep, batteries, memory pressure and a watcher that would not restart unasked each stopped work until I acted.

This paper is Claude's reading of the records. Its choice of illustrations is a selection.

Lessons. Write the stop rule and the pass criteria before the run, and keep the control runnable. Save every input a failure depends on. Give every new value a second path on another machine. Prefer a fail-fast to a long wait, and tie timing rules to what the process is doing. Let runners amend a rule they can see is wrong. Keep one session writing the code, and state each standing instruction once, where every session reads it.

XI. HOW THIS WORK WAS DONE

I am the author of this work and of this paper. Claude, Anthropic's model, used through Claude Code, was the tool. It produced the code, the computations and the notes, as a coordinating session, runner sessions and subagents under my direction. This paper was drafted by a Claude subagent from the chat where the project began, the files and the transcripts named in `paper/ai/SOURCES.md`, and shortened at my request after I read the first draft. Its numbers come from files and from commands run on them. The references were checked against publisher, catalogue or repository records, and the one that could be checked only against a citing record is marked. No human expert has reviewed it. Corrections are welcome.

Data and code. https://github.com/nmoinvaz/choptuik-large-d and https://github.com/nmoinvaz/choptuik-large-d-raw.

[1] M. W. Choptuik, "Universality and scaling in gravitational collapse of a massless scalar field", Phys. Rev. Lett. **70**, 9 (1993), doi:10.1103/PhysRevLett.70.9.

[2] N. Moinvaziri, `choptuik-large-d`, https://github.com/nmoinvaz/choptuik-large-d (2026).

[3] N. Moinvaziri, "Choptuik's echo period and critical exponent at large dimension: evidence for $\Delta_\infty = 1.643893746$ and an r^2-shift symmetry behind $\gamma_\infty = 1/2$", draft, `paper/physics/dss_physics.pdf` in Ref. [2].

[4] N. Moinvaziri, "The echo period of Choptuik's critical solution at large dimension: a selection rule and a computer-assisted proof", draft, `paper/math/dss_math.pdf` in Ref. [2].

[5] T. J. Dekker, "A floating-point technique for extending the available precision", Numer. Math. **18**, 224 (1971), doi:10.1007/BF01397083.
[6] Y. Hida, X. S. Li and D. H. Bailey, "Algorithms for quad-double precision floating point arithmetic", in *Proc. 15th IEEE Symposium on Computer Arithmetic (ARITH-15)*, 155–162 (2001).
[7] Anthropic, Claude Code documentation, https://code.claude.com/docs; on subagents, https://code.claude.com/docs/en/sub-agents (read 6 November 2026).
[8] N. Moinvaziri, `choptuik-large-d-raw`, https://github.com/nmoinvaz/choptuik-large-d-raw (2026).
[9] N. Moinvaziri, the chat on claude.ai where the project began, 17–19 October 2026 (UTC), unpublished.
[10] N. Moinvaziri, `multidouble`, https://github.com/nmoinvaz/multidouble.
[11] M. D. Schwartz, BootLoops, https://github.com/BootLoops-ai/bootloops (read at commit e16ba28 on 21 October 2026).
[12] py-spy, a sampling profiler for Python programs, https://github.com/benfred/py-spy.
[13] B. Cochelin, "A path-following technique via an asymptotic-numerical method", Comput. Struct. **53**, 1181 (1994) (checked against a citing record only).